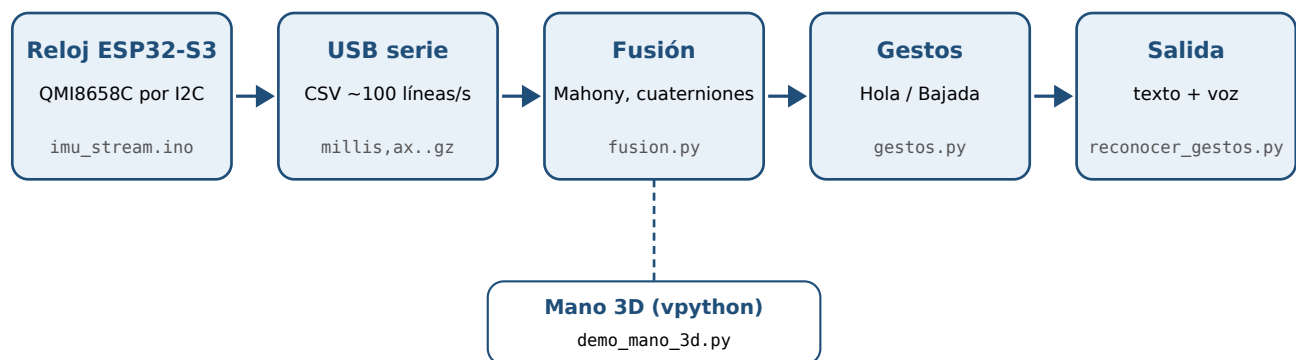


Guante Traductor — Versión 1.2

Procesos matemáticos, de programación y de hardware, explicados según cómo se usan en el programa

Este documento describe **qué cálculos y qué técnicas usa el proyecto y para qué sirven dentro del código**. No es un curso: cada tema se explica solo hasta donde hace falta para entender las líneas del programa, con los valores reales que usa la versión 1.2 (rama `claude/hopeful-hopper-eycthu`, commit `8680ac9`). Los nombres de funciones y variables coinciden con los archivos de `orientacionMano_visual-V1.2/`.

Recorrido de un dato, de la muñeca a la palabra



El reloj solo mide y envía números crudos convertidos a unidades. Toda la matemática (orientación, gestos, voz) corre en Python en la computadora; así se ajusta sin volver a programar el reloj.

1. Hardware y firmware: obtener los números

1.1 Hablar con el sensor por I2C

El IMU QMI8658C comparte un bus I2C con otros chips del reloj (pantalla táctil, cargador, reloj de tiempo real). I2C usa dos cables: **SDA** (datos, GPIO 15) y **SCL** (reloj del bus, GPIO 14). Cada chip tiene una **dirección**; para leer un dato se envía la dirección del chip, luego el número de registro, y el chip responde con los bytes de ese registro.

- **Detección de la dirección:** `detectarIMU()` prueba `0x6B` y `0x6A` y lee el registro `0x00` (`WHO_AM_I`). Solo acepta la que responde `0x05`, que es la firma del QMI8658C. Si ninguna responde, el `loop()` envía cada segundo la línea `# IMU no detectado` (Python la ignora por empezar con `#`).
- **Velocidad del bus:** `Wire.setClock(400000)` = 400 kHz (modo rápido), suficiente para leer 12 bytes cientos de veces por segundo.
- **Lectura con reinicio:** `endTransmission(false)` no suelta el bus entre “quiero el registro `0x35`” y “dame 12 bytes”, para que otro chip no se meta en medio.

1.2 Configuración por registros

Registro	Valor	Qué hace en este programa
0x02 CTRL1	0x40	Auto-incremento: al pedir 12 bytes desde 0x35 el chip avanza solo por AX, AY, AZ, GX, GY, GZ.
0x03 CTRL2	0x16	Acelerómetro en ± 4 g y 125 muestras/s.
0x04 CTRL3	0x56	Giroscopio en ± 512 °/s y 125 muestras/s.
0x08 CTRL7	0x03	Enciende acelerómetro y giroscopio.

1.3 De bytes a unidades físicas

Cada eje llega como dos bytes, primero el bajo y luego el alto (little-endian). Se unen en un entero de 16 bits **con signo** (complemento a dos, rango -32768 a 32767):

```
int16_t ax = (int16_t)(b[1] << 8 | b[0]); // byte alto desplazado 8 bits + byte bajo
```

El rango completo de 16 bits representa el rango elegido en la configuración, de ahí los factores de escala:

```
acelerómetro: 32768 cuentas / 4 g      = 8192 cuentas por g      -> ax / 8192.0
giroscopio:   32768 cuentas / 512 °/s = 64 cuentas por °/s     -> gx / 64.0
```

Ejemplo. si llegan los bytes $b[0]=0x00$, $b[1]=0x20$, el entero es $0x2000 = 8192$ y $8192/8192 = 1.00$ g: es la gravedad cuando ese eje apunta hacia arriba. Si llegara $0xE000$, el $(int16_t)$ lo interpreta como -8192 , es decir -1 g.

1.4 Protocolo por USB

El ESP32-S3 tiene USB nativo (no hay chip convertidor), por eso hace falta **USB CDC On Boot = Enabled**: así Serial sale por el conector USB-C. Cada muestra es una línea de texto:

```
123456,0.0123,-0.0210,0.9981,0.512,-0.300,0.094
millis ax      ay      az      gx      gy      gz
```

- millis es el tiempo del reloj en milisegundos. Python lo usa para calcular el intervalo real entre muestras (dt), en lugar de suponer que siempre es 10 ms.
- delay(10) más el tiempo de lectura dan un poco menos de 100 muestras por segundo.
- Texto en vez de binario: más lento, pero se puede leer a simple vista en el Monitor Serie y cualquier línea rota se descarta sin desalinear las siguientes.

2. Lectura en Python y calibración

2.1 Filtrado de líneas

leer_muestras() vacía todo lo que haya en el búfer del puerto (in_waiting), separa por comas y acepta la línea solo si tiene exactamente 7 números. Así se descartan los mensajes de diagnóstico y las líneas cortadas a la mitad al abrir el puerto.

2.2 El paso de tiempo dt

```
t = t_ms / 1000.0
if t_prev is not None and 0 < t - t_prev < 0.5:
    q = actualizar(q, a, g, t - t_prev)
```

Solo se integra si dt es positivo y menor que medio segundo. Un dt negativo o enorme indica que el reloj se reinició o que se perdieron datos; integrarlo haría girar la mano de golpe.

2.3 Sesgo del giroscopio (promedio)

Un giroscopio quieto no marca exactamente 0 °/s, sino unos pocos grados por segundo (sesgo). Como la orientación se obtiene **sumando** giro por tiempo, un sesgo de 2 °/s acumularía 120° en un minuto. Al arrancar se promedian 200 muestras con el reloj quieto (unos 2 s) y ese promedio se resta siempre:

```
sesgo = suma / n                # promedio de gx, gy, gz en reposo
g = np.array([gx, gy, gz]) - sesgo # giro "limpio" en °/s
np.radians(g)                   # la fusión trabaja en rad/s
```

Consecuencia práctica. si el reloj se mueve durante la calibración, el sesgo queda mal y la mano virtual gira sola lentamente. La solución es reiniciar el programa con el reloj apoyado.

3. Orientación con cuaterniones (fusion.py)

3.1 Por qué cuaterniones y no ángulos

La orientación del reloj se podría guardar como tres ángulos (alabeo, cabeceo, guiñada), pero con la muñeca girada cerca de 90° dos de esos ejes se alinean y el cálculo se vuelve inestable (**bloqueo de cardán**): la versión anterior hacía que la mano “se volviera loca”. Un cuaternión guarda la misma orientación con cuatro números y no tiene ese punto problemático.

```
q = [w, x, y, z]      # empieza en [1, 0, 0, 0] = sin rotación
```

Para este programa basta saber que:

- (x, y, z) apunta en la dirección del eje de giro y w depende de cuánto se ha girado.
- Solo representan una rotación si su longitud es 1; por eso el código normaliza en cada paso ($q / \text{np.linalg.norm}(q)$). Sin eso, los errores de redondeo irían deformando la mano.
- Nunca se convierten a ángulos: se convierten a una **matriz de rotación** (sección 3.4) para usarlos.

3.2 Integrar el giroscopio

El giroscopio dice qué tan rápido gira el reloj alrededor de sus propios ejes, $\omega = (g_x, g_y, g_z)$ en rad/s. La regla para actualizar el cuaternión es:

```
dq/dt = 1/2 * q ⊗ (0, ωx, ωy, ωz)      (⊗ = producto de cuaterniones)

dq = 0.5 * [ -x*gx - y*gy - z*gz,
             w*gx + y*gz - z*gy,
             w*gy - x*gz + z*gx,
             w*gz + x*gy - y*gx ]
q = q + dq * dt      # paso de Euler: "cambio por segundo" × segundos transcurridos
q = q / |q|          # volver a longitud 1
```

Es la versión en cuaterniones de “ángulo = velocidad × tiempo”. El paso de Euler es la forma más simple de integrar; con $dt \approx 10$ ms el error es despreciable frente al ruido del sensor.

3.3 Corregir la deriva con el acelerómetro (filtro Mahony)

Integrar solo el giroscopio acumula error. El acelerómetro, en reposo, mide la gravedad y por lo tanto sabe dónde está “arriba”. El filtro compara las dos opiniones y corrige al giroscopio:

```
a = a / |a| # "arriba" medido por el acelerómetro
v = [2(xz - wy), 2(wx + yz), w2 - x2 - y2 + z2] # "arriba" según el cuaternión
e = a × v # producto cruz = error de orientación
g = g + KP * e # se suma como giro extra de corrección
```

- **v** es la tercera fila de la matriz de rotación: el eje vertical del mundo expresado en ejes del chip. Es lo que el acelerómetro debería medir si el cuaternión fuera correcto.
- **Producto cruz a × v**: si ambas direcciones coinciden da cero (no hay nada que corregir). Si no, da un vector perpendicular a las dos, que es justo el eje alrededor del cual hay que girar para alinearlas, y su tamaño crece con el ángulo entre ellas.
- **KP = 2.0** es cuánto se cree al acelerómetro. Más alto corrige más rápido la deriva, pero la mano tiembla más porque los movimientos bruscos también empujan al acelerómetro. Con KP = 2 el error se reduce a la mitad en unos 0.35 s ($\ln 2 / KP$).
- Se normaliza **a** porque solo importa la dirección; así da igual si en ese instante mide 0.9 g o 1.1 g.

Límite importante. el acelerómetro no puede ver giros alrededor del eje vertical (la gravedad no cambia al girar sobre ella). Por eso la inclinación (alabeo y cabeceo) se mantiene estable, pero la dirección horizontal (guiñada) se desvía lentamente. Se necesitaría un magnetómetro para corregirla, y este IMU no lo tiene.

3.4 Matriz de rotación

matriz(q) convierte el cuaternión en una matriz 3×3 **R**. Multiplicar **R · v** toma un vector expresado en ejes del chip y lo devuelve en ejes del mundo (x, y horizontales; z hacia arriba). Así se sabe hacia dónde apuntan los dedos en la habitación.

```
R = [[1-2(y2+z2), 2(xy-wz), 2(xz+wy) ],
      [2(xy+wz), 1-2(x2+z2), 2(yz-wx) ],
      [2(xz-wy), 2(yz+wx), 1-2(x2+y2) ]]
```

3.5 Montaje del reloj: HAND_FWD y HAND_UP

El programa no sabe cómo está puesto el reloj en la mano. Se le indica con dos vectores en ejes del chip (en config.py, compartidos por todos los scripts):

- HAND_FWD = [0, -1, 0]: el eje del chip que apunta hacia los dedos.
- HAND_UP = [0, 0, 1]: sale de la pantalla, es decir, el dorso de la mano.

La tecla **i** de la demo 3D invierte el signo de HAND_FWD e imprime la línea para copiar a config.py.

3.6 Elevación de los dedos

```
f = R · HAND_FWD # dirección de los dedos en el mundo (vector de largo 1)
elev = asin(f_z) (en grados) # f_z = componente vertical
```

Como f mide 1, su componente vertical es el seno del ángulo con la horizontal: $+90^\circ$ dedos al techo, 0° horizontales, -90° al piso. El valor se limita a $[-1, 1]$ antes del arcoseno porque el redondeo puede dar 1.0000001 y eso rompería la función. Este número es la única entrada del detector de “Yo soy”.

4. Visualización 3D (demo_mano_3d.py)

La mano es un compound de cajas (palma, cuatro dedos, pulgar) construido con los dedos a lo largo del eje x. Para orientarla, vpython solo necesita dos vectores: axis (hacia dónde apunta el objeto) y up (qué lado es arriba):

```
mano.axis = a_vpython(R @ HAND_FWD)
mano.up    = a_vpython(R @ HAND_UP)
```

- **Cambio de ejes:** en el mundo del programa z es arriba, pero en vpython arriba es y. $a_vpython(v)$ devuelve $(v_x, v_z, -v_y)$; el signo menos mantiene la regla de la mano derecha para que la mano no aparezca reflejada.
- **Recentrar (tecla z):** calcula el rumbo horizontal actual de los dedos con $yaw0 = \text{atan2}(f_y, f_x)$ y desde entonces aplica $R_z(-yaw0)$, una rotación alrededor del eje vertical que lleva ese rumbo al frente. Compensa la deriva de guiñada de la sección 3.3 sin tocar la inclinación.
- `rate(200)` limita el ciclo a 200 vueltas por segundo para no ocupar todo el procesador.

5. Detección de gestos (gestos.py)

Los detectores son **reglas**, no aprendizaje automático. Cada uno recibe las muestras en orden, guarda solo lo necesario de los últimos instantes y devuelve True cuando se cumple su regla.

5.1 “Hola”: contar vaivenes del giroscopio

Al saludar, la mano va y viene, así que el giroscopio cambia de signo una y otra vez. DetectorHola convierte cada eje en un estado de tres valores y cuenta los cambios:

```
s = +1 si g > 90 °/s
s = -1 si g < -90 °/s
s = 0 en medio -> se ignora, se conserva el último signo

si s cambia de +1 a -1 (o al revés): se guarda el instante en una cola
se borran de la cola los instantes con más de 1.6 s
si la cola de algún eje llega a 3 cambios -> "Hola"
```

- **Zona muerta ($\pm 90^\circ/\text{s}$):** el temblor o el ruido nunca cruzan de un lado al otro, así que no cuentan. Es una histéresis: para contar un cambio hay que ir claramente de un sentido al contrario.
- **Ventana deslizante con deque:** una cola donde se agrega por un lado y se quita por el otro; mantiene solo los eventos recientes con costo mínimo.
- **Los tres ejes por separado:** basta que cualquiera cumpla, así funciona sin importar cómo esté montado el reloj.
- **Enfriamiento de 2 s:** tras detectar, ignora todo durante 2 s para no decir “Hola” varias veces en un mismo saludo.

Números de la prueba. en la simulación, un saludo de 2.5 vaivenes por segundo y $\pm 40^\circ$ de amplitud llega a $40^\circ \times 2\pi \times 2.5 \approx 628$ $^\circ/\text{s}$ en su punto más rápido, muy por encima del umbral de 90 $^\circ/\text{s}$. Se detecta a los 0.51 s, al completar el tercer cambio de sentido.

5.2 “Yo soy”: caída rápida de la elevación

```
historial = [(t, elev), ...] de los últimos 0.6 s
si max(elev en el historial) - elev_actual >= 30° -> "bajada"
```

Compara la elevación actual con la más alta de los últimos 0.6 s. Es una forma directa de medir **velocidad de caída**: 30° en 0.6 s equivale a por lo menos 50 $^\circ/\text{s}$ sostenidos. Una bajada lenta (50° en 3 s) nunca junta 30° dentro de una ventana de 0.6 s, por eso no dispara. Enfriamiento: 1.5 s.

5.3 Reconocedor: resolver el conflicto entre ambos

El saludo también baja la mano, y la bajada ocurre antes de completar los tres vaivenes. Sin cuidado, el programa decía “Yo soy” y luego “Hola”. La clase Reconocedor aplica dos reglas de tiempo:

- **Confirmación diferida (0.8 s):** una bajada queda pendiente. Si en los siguientes 0.8 s aparece un “Hola”, se descarta; si no, se confirma. Es una pequeña máquina de estados: sin pendiente → pendiente → (confirmada | descartada).
- **Silencio tras saludar (1 s):** las bajadas en el segundo posterior a un “Hola” se ignoran; son el rebote de bajar la mano después de saludar.

El costo es que “Yo soy” suena unos 0.8 s después del gesto. Es un compromiso entre rapidez y no confundir gestos, ajustable con `espera_s`.

Parámetro	Valor	Qué pasa si se cambia
umbral_dps (Hola)	90 $^\circ/\text{s}$	Más alto: menos falsos “Hola”, pero hay que saludar con más energía.
cambios (Hola)	3	Más: exige un saludo más largo.
ventana_s (Hola)	1.6 s	Más corta: exige saludar más rápido.
caida_deg (Bajada)	30°	Más alta: hay que bajar más la mano.
ventana_s (Bajada)	0.6 s	Más corta: la bajada tiene que ser más rápida.
espera_s (Reconocedor)	0.8 s	Más corta: responde antes, pero puede confundir con el saludo.
tras_hola_s (Reconocedor)	1.0 s	Tiempo en que se ignoran bajadas después de “Hola”.

6. Salida de texto y voz (reconocer_gestos.py)

`decir(texto)` imprime siempre la palabra y, si `pyttsx3` está instalado, la manda a hablar. Hablar tarda alrededor de un segundo, y en ese tiempo el programa no puede quedarse sin leer el puerto. Por eso se usa un patrón **productor-consumidor**:

```
hilo principal: lee IMU -> detecta gesto -> _frases.put("Hola")    (no espera)
hilo de voz:     texto = _frases.get()    -> pyttsx3 habla -> espera la siguiente
```

- **Cola (queue.Queue):** es segura entre hilos. Si se detectan dos gestos seguidos, las frases esperan su turno en orden.
- **Hilo daemon:** se cierra solo al terminar el programa.
- **Motor nuevo por frase:** pyttsx3 en Windows a veces deja de hablar después de la primera frase si se reutiliza el motor. Crearlo en cada frase evita ese fallo conocido.
- **COM en el hilo:** las voces de Windows (SAPI) usan COM, que debe inicializarse en el hilo que las usa (comtypes.CoInitialize()).

Pendiente. la voz todavía no se ha probado en la PC real, y el código no elige una voz en español, así que se usa la voz predeterminada del sistema.

7. Pruebas simuladas (test_sim.py)

Para probar sin el reloj se simula un **cuerpo rígido**: se inventa una velocidad de giro en el tiempo, se integra con la misma ecuación de la sección 3.2 para obtener la orientación “real”, y a partir de ella se fabrican las lecturas que daría el sensor:

```
giroscopio simulado = la velocidad de giro inventada
acelerómetro simulado =  $R^T \cdot [0, 0, 1]$  # la gravedad vista desde el chip
```

R^T (la transpuesta) hace la conversión inversa a R : de ejes del mundo a ejes del chip. Luego esas lecturas pasan por el filtro y los detectores reales, igual que si vinieran del USB.

- **Ejes derivados de HAND_FWD:** el eje que baja los dedos se calcula como $BAJAR = \hat{z} \times HAND_FWD$ (producto cruz con la vertical). Si se cambia el montaje en config.py, las pruebas siguen siendo válidas sin reescribirlas.
- **Ruido:** el caso “quieto” agrega ruido aleatorio con semilla fija (default_rng(0)), así el resultado es el mismo en cada ejecución.
- **Verificación:** cada caso compara los gestos detectados con los esperados, imprime OK o FALLA y el programa termina con código 1 si algo falla.

Caso simulado	Esperado	Resultado
Quieto con ruido (5 s)	nada	nada
Saludo 2.5 Hz, $\pm 40^\circ$	hola	hola a 0.51 s
Bajada 125 °/s durante 0.4 s (50°)	bajada	bajada a 2.04 s (1.24 + 0.8 de espera)
Bajada lenta, 50° en 3 s	nada	nada
Un giro de muñeca de 90°	nada	nada
Bajada justo antes de saludar	solo hola	hola a 1.81 s (la bajada se descarta)

Qué no demuestran. son movimientos idealizados. Una persona real saluda con otra amplitud y velocidad, y el acelerómetro también mide los empujones del movimiento (no solo la gravedad). Los umbrales deben ajustarse con el reloj puesto usando reconocer_gestos.py --debug, que cada segundo imprime la elevación y el pico del giroscopio por eje.

8. Resumen: cada técnica y su papel

Técnica	Dónde	Para qué se usa aquí
I2C + registros	imu_stream.ino	Encontrar y configurar el IMU, leer 12 bytes por muestra.
Complemento a dos, escala	imu_stream.ino	Convertir bytes en g y °/s.
CSV por USB CDC	firmware + Python	Protocolo simple, legible y tolerante a líneas rotas.
Promedio (sesgo)	calibración	Quitar el error fijo del giroscopio.
Cuaterniones	fusion.py	Guardar la orientación sin bloqueo de cardán.
Integración de Euler	fusion.py	Sumar giro × tiempo en cada muestra.
Producto cruz + KP	fusion.py	Corregir la deriva con la gravedad (Mahony).
Matriz de rotación	fusion.py	Pasar direcciones del chip al mundo.
Arcoseno	elevacion_deg	Ángulo de los dedos con la horizontal.
atan2 + Rz	demo_mano_3d.py	Recentrar la dirección horizontal (tecla z).
Histéresis + ventana deslizante	DetectorHola	Contar vaivenes ignorando el ruido.
Máximo en ventana	DetectorBajada	Medir caídas rápidas de la elevación.
Máquina de estados con espera	Reconocedor	No confundir la bajada del saludo con “Yo soy”.
Hilo + cola	reconocer_gestos.py	Hablar sin detener la lectura del reloj.
Simulación de cuerpo rígido	test_sim.py	Probar todo sin hardware.